

MATERIAL EXTRA

Padronização de Projetos



A IMPORTÂNCIA DA PADRONIZAÇÃO DE PROJETOS

Escrever código é uma atividade que vai muito além de simplesmente fazer um programa funcionar. Um código bem escrito precisa ser legível, organizado e de fácil manutenção. Esse aspecto se torna ainda mais relevante quando o projeto é desenvolvido por uma equipe, onde cada desenvolvedor pode ter um estilo de escrita diferente. Para evitar inconsistências e garantir um código padronizado, ferramentas como ESLint e Prettier desempenham um papel fundamental.

A falta de padronização pode gerar dificuldades na hora de dar manutenção no código, aumentando a possibilidade de bugs e tornando mais complexo o processo de revisão de código (code review). Imagine um projeto onde cada desenvolvedor segue suas próprias regras de formatação, espaçamento, nomeação de variáveis e indentação. O código pode se tornar um verdadeiro caos, dificultando a leitura e o entendimento do que está sendo feito. Isso impacta diretamente na produtividade e na qualidade do software.

O Papel do ESLint

O ESLint é uma ferramenta de análise estática de código que ajuda a identificar e corrigir problemas no JavaScript e no TypeScript. Ele funciona verificando o código em busca de padrões que possam indicar erros, más práticas ou inconsistências. Além disso, o ESLint permite definir regras personalizadas para adequar-se às necessidades do projeto, garantindo que todos os desenvolvedores sigam um conjunto de diretrizes bem definido.

Entre os principais benefícios do ESLint, podemos destacar:

- **Deteção de erros comuns:** O ESLint pode evitar bugs ao alertar sobre problemas como variáveis não utilizadas, uso incorreto de `await`, funções assíncronas sem `async`, entre outros.
- **Adoção de boas práticas:** A ferramenta permite configurar regras que seguem padrões de qualidade, como evitar o uso de `var` e preferir `const` ou `let`.
- **Padronização do código:** Mantém um estilo de código consistente entre todos os desenvolvedores do time.

A configuração do ESLint pode ser ajustada conforme as necessidades do projeto. Ao instalar e configurar corretamente, ele pode até mesmo corrigir automaticamente alguns erros detectados.

O Papel do Prettier

Se o ESLint é responsável por encontrar problemas no código, o Prettier entra em cena para garantir uma formatação consistente. Ele padroniza a estrutura do código, aplicando automaticamente regras como espaçamento, indentação, uso de aspas e quebras de linha. Isso significa que, ao salvar um arquivo, o código será reformatado de acordo com as regras definidas, garantindo uniformidade em toda a base de código.

Os principais benefícios do Prettier incluem:

Evita discussões sobre estilo de código: Ao adotar uma ferramenta automatizada para formatação, a equipe não precisa debater sobre detalhes como tamanho de indentação ou estilo de aspas.

Melhora a leitura do código: Com um código sempre bem organizado, a compreensão do fluxo da aplicação se torna mais rápida e intuitiva.

Aumenta a produtividade: Com a formatação automática, os desenvolvedores podem se concentrar na lógica do código, sem perder tempo ajustando espaçamentos manualmente.

Assim como o ESLint, o Prettier pode ser configurado para atender às necessidades específicas do projeto. Ele também pode ser integrado ao ESLint para que ambas as ferramentas funcionem juntas sem conflitos.

Integrando ESLint e Prettier no Projeto

Para garantir uma padronização completa, o ideal é utilizar ESLint e Prettier juntos. A configuração integrada evita que haja conflitos entre as regras de linting e as de formatação automática. O processo de configuração geralmente envolve os seguintes passos:

1. Instalar as dependências necessárias:

```
npm install --save-dev eslint prettier eslint-config-prettier  
eslint-plugin-prettier
```

2. **Configurar o ESLint:** Após a instalação, é necessário criar um arquivo de configuração `.eslintrc.json` na raiz do projeto e definir as regras desejadas.

```
{
  "extends": ["eslint:recommended", "plugin:prettier/recommended"],
  "rules": {
    "prettier/prettier": "error",
    "no-unused-vars": "warn",
    "semi": ["error", "always"]
  }
}
```

3. **Configurar o Prettier:** Criar um arquivo `.prettierrc` para definir as regras de formatação.

```
{
  "singleQuote": true,
  "trailingComma": "es5",
  "tabWidth": 2,
  "semi": true
}
```

4. **Automatizar a formatação ao salvar:** Se estiver utilizando o VS Code, é recomendável instalar as extensões do ESLint e do Prettier e ativar a opção de formatação automática ao salvar arquivos.

Isso pode ser feito adicionando a seguinte configuração ao arquivo **settings.json** do VS Code.

```
{
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  }
}
```

Com essa configuração, sempre que um arquivo for salvo, o Prettier aplicará a formatação automática e o ESLint corrigirá possíveis erros automaticamente.

Conclusão

A adoção do ESLint e do Prettier nos projetos traz benefícios significativos para o desenvolvimento de software. A padronização do código melhora a legibilidade, reduz erros e aumenta a produtividade da equipe. Com um código bem estruturado e formatado de maneira consistente, as revisões se tornam mais ágeis e as chances de introdução de bugs diminuem.

Para desenvolvedores iniciantes, a utilização dessas ferramentas desde o começo ajuda a criar bons hábitos e a seguir boas práticas da indústria. Para times experientes, elas representam um ganho de tempo e qualidade, permitindo que os desenvolvedores foquem na lógica da aplicação em vez de se preocuparem com detalhes de formatação e regras de sintaxe.

Independentemente do tamanho do projeto ou da equipe, investir na padronização do código utilizando ESLint e Prettier é uma decisão inteligente que traz retornos positivos a longo prazo. Portanto, se você ainda não utiliza essas ferramentas, experimente configurá-las no seu próximo projeto e aproveite os benefícios que elas oferecem!



BONS ESTUDOS

